

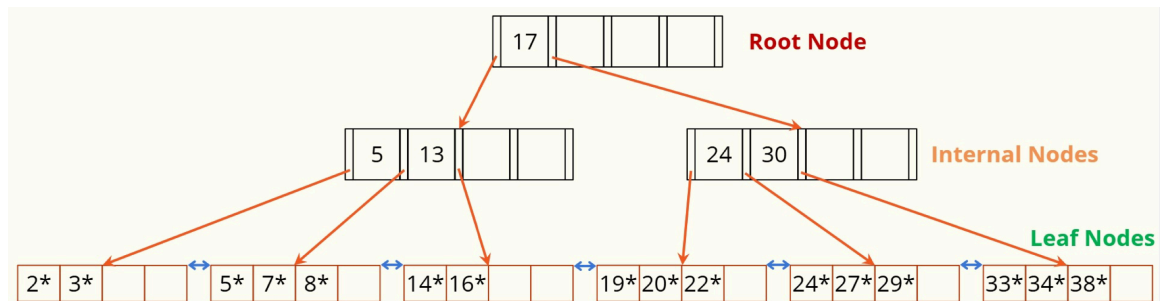
09 B-Tree indexes, JSON

▼ B+ Tree Index

General

- A balanced tree data structure
- Most common index type
- Supports both point and range queries

▼ Sample B+ Tree Index

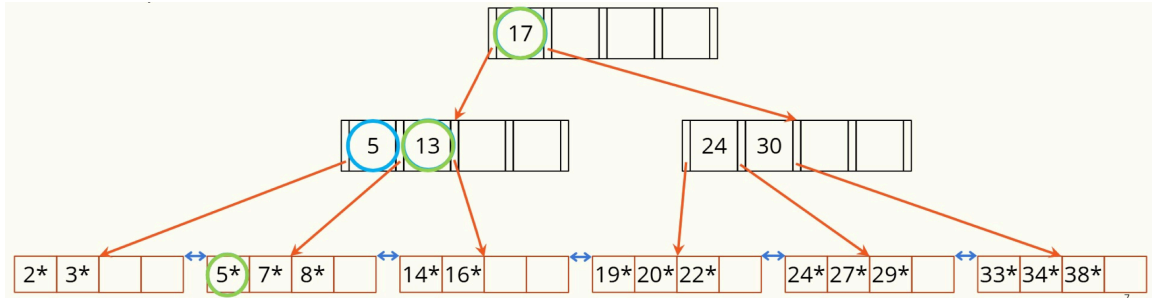


- Leafs have a **maximum and minimum capacity (Order)**
- Leafs are connected through a **linked list**
- Each node is **one disk page**
- A node can consists of **several keys**

▼ Searching

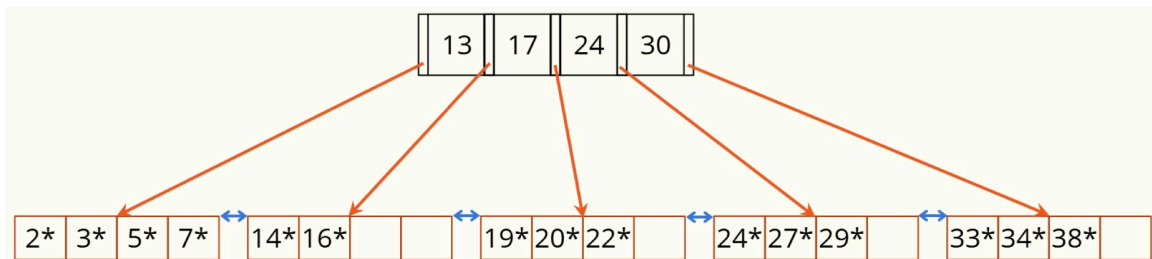
- Comparing **search key (SK)** against node **key element (K)**
 - If $SK < K$ go to the left child node of K
 - Else check K+1
 - If the final element is reached and is still not passing the statement, go to the right-most child node

Example: Find 5*

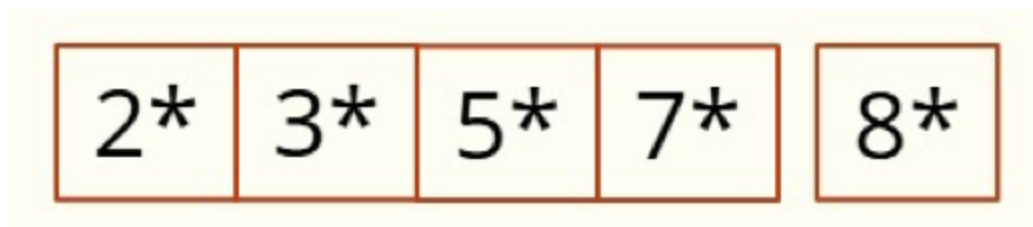


▼ Inserting and splitting

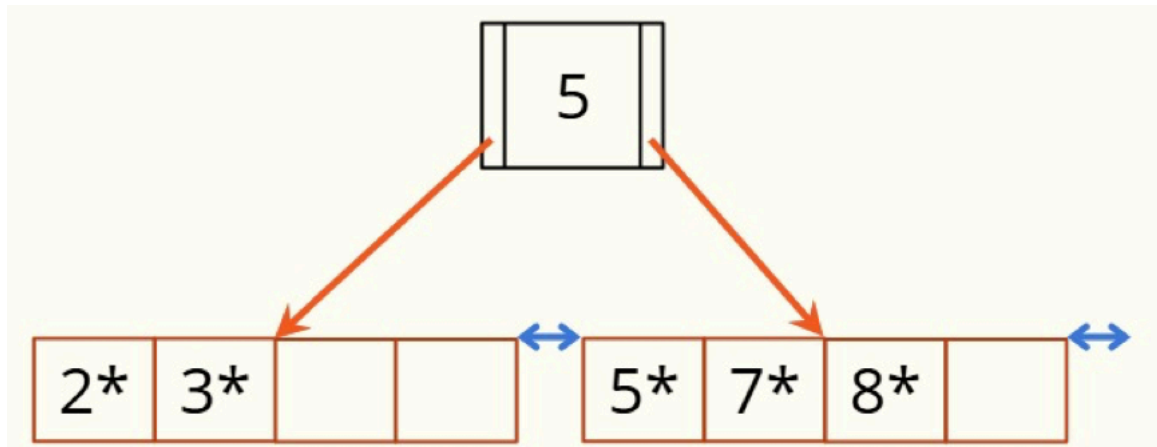
Example: insert 8*



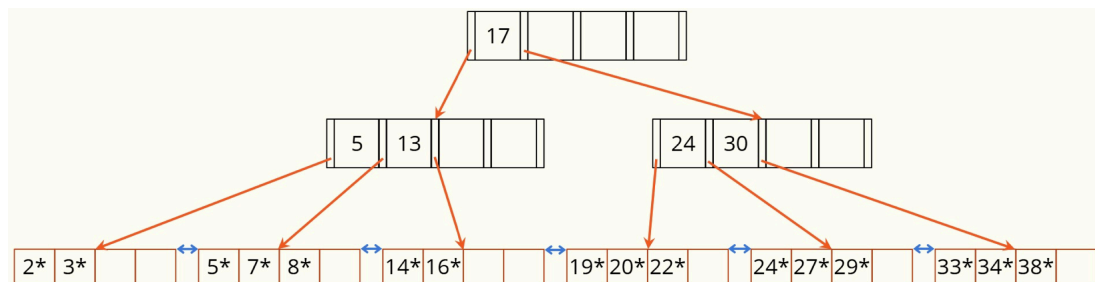
- Start by using the search strategy, to find the appropriate leaf node
 - Leaf-node left is full, and so is leaf-node right.
 - Therefore we must split
- When splitting, we **assume that the new elements have been inserted**:



- The middle element becomes the parent of the new nodes



- The tree now looks like this:



▼ Storage

General:

- Order: 1000 (≈ 16 KB per page / 16 bytes per entry)
- Utilization is about 67% in practice
- Fanout: 670

Capacity:

- Root = 670 records
- Two levels = 670^2 records = 448.900 records
- Three levels = 670^3 records = 300.763.000 records

Memory:

- Level 1 = 1 page = 16 KB
- Level 2 = 670 pages = 11 MB
- Level 3 = 448.900 pages = 7 gb

▼ JSON

General

- A semi-structured data structure
- Human readable and easy to interpret format
- Commonly used for **transmitting data** between a **server** and **web applications**

▼ Document Databases

- **Flexibility** (schemaless) vs **Rigid** (enforced schemas)
 - Understand your **business requirements**
 - Being schemaless can be dangerous if not kept in check
- Types of **NoSQL databases**:
 - Document Databases
 - Graph Databases
 - Key-Value Databases
- **NoSQL databases excel in specialized cases**:
 - Handling **large volumes of unstructured** data
 - Horizontal scaling
 - Real-time analytics

▼ JSON in DuckDB

Reading a JSON file:

- Simple SELECT Statement:

```
SELECT * FROM 'my_file.json';
```

- For better transparency and additional features use the function `read_json()`

```
SELECT * FROM read_json('my_file.json')
```

- Specify which columns (keys) to get

```
SELECT * FROM read_json('my_file.json', columns={key_1: 'DataType', k
```

Reading JSON into table:

```
CREATE TABLE json_test AS SELECT * FROM read_json('examples.json
```